Assignment 3 – Standard Template Library [Part 1]

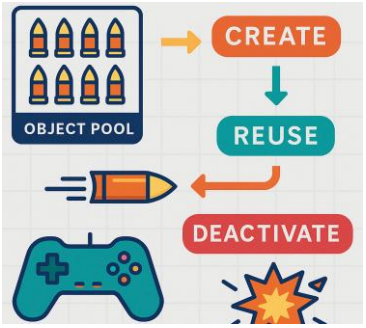
Goal

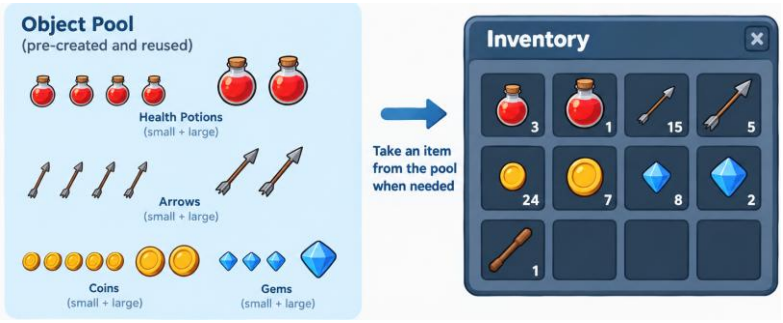
The goal of this assignment is to put into practice the use of containers, iterators, and algorithms provided by the Standard Template Library (STL). Students will implement an object pooling mechanism, a fundamental building block of modern game architectures. Through this assignment, students will obtain experience in the design and implementation of an object pool.

Description

An object pool is a mechanism that allows objects to be reused rather than repeatedly created and destroyed. Creating and destroying objects at runtime can be computationally expensive, particularly in fast-paced games. Object pooling can improve performance by pre-allocating a collection of objects and deactivating and reactivating them as needed.

This technique can reduce runtime overhead, improve memory management, and minimize garbage collection overhead. Examples of objects that can benefit from pooling include bullets and inventory items.





The objective of this assignment is to extend the Entity-Component framework for game development to implement an object pooling mechanism.

1. Pool Item class (pool_item.hh)

You must implement a **`PoolItem`** class, which represents an item stored in the pool for the purposes of this assignment. A **`PoolItem`** has a name and provides methods to activate and deactivate the item for convenience. The class inherits from the abstract **`Entity`** class and it is partially provided.

The class must meet the following requirements:

Attributes	Description

Name (string)	Name and unique identifier of the Item. Its visibility must be public.
Active (bool)	Flag that indicates if the item is active or inactive. Its visibility must be public.
Methods	**Description**
Constructor	The constructor of the class with its assigned name and state (active or inactive)
update	Overload of the update method from Entity class (provided)
draw	Overload of the draw method from Entity class (provided)
Other methods	You may need to implement additional methods for this class for the pool

2. Object pool (object_pool.hh):

The class is responsible for managing a collection of items (objects of type **`PoolItem`).** It must provide a set of useful functions, including adding items to a container, finding items, activating and deactivating items by name, counting active and inactive objects, updating and drawing active items, among other operations.

At first glance, this may seem straightforward to implement. However, the main challenge is that using **iterator-based loops are not allowed**. Instead, **you must use only the algorithms provided by the Standard Template Library (STL), together with Object Functions or Functors**.

The class must fulfill the following requirements:

Attributes	**Description**
Object container	STL Container of objects of type PoolItem
Methods	**Description**
add	Add an item to the container.
count	Count all the elements in the container.
countActive	Count all the elements that are active.
findByName	Find an entity by name. It must return the address of the first item found with the name. To obtain the address of an iterator, you need to dereference the element first: &(*it). If not item with the name is found, the method must return nullptr.
findByRef	Method to find an item using other item as reference. If not equivalent item is found must return nullptr.
activate	Activate an item by its name, it must set the property active=true
deactivate	Deactivate an item by its name, it must set the property active=false
update	Calls the update method for all the items. It must call the update method from the PoolItem class for each item.
updateActive	Call the update method for only the active items.
draw	Call the draw method for all the items. It must call the draw method from the PoolItem class for each item.
drawActive	Call the draw method for only the active entities.
remove	Remove an item by name from the container and releases its memory
removeInactive	Remove all inactive items from the container and releases its memory

The primary objective of this first part is to ensure that all tests compile and pass successfully. Code quality, documentation, and overall organization will also be evaluated. Students should not modify any files related to the tests; only the files specified for the assignment can be changed. Upon completion, students should have implemented a usable framework capable of object pooling.

What to do:

The code explicitly requires the following files, which the student should create and submit:

- pool_item.hh, pool_item.cc, object_pool.hh, object_pool.cc

Submission Guidelines

Submit all the files as a zip folder on Moodle with the following naming conventions:
cs225fa26_{username}_3.1.zip